

Gesture Driven Generative Canvas

Author: Yuna Mishra

System Architecture: Computer Vision, Machine Learning, and Real-Time Graphics
Rendering Pipeline

Gesture Driven Generative Canvas	1
Project Overview	2
Motivation & Objective	2
Architecture & Requirements	3
Tools & Libraries	3
Concepts & Math Used	3
How It Works (Execution Flow)	4
Setup & How to Run	5
Issues Faced & Solutions	6
AI Assistance & Technical Lessons	6

Project Overview

Motivation & Objective

Looking to break away from the regular (and let's be honest, amazing) routine of life and stay in this AI world, I decided to build something ridiculous, fun, and visual using computer vision.

The goal was to track my hand gestures through a webcam and create "Wanda-style" magical visual effects on the screen—like glowing rings, particle trails, and energetic canvas drawings—in real time. Instead of relying on hardcoded distance thresholds that break easily, I trained a machine learning model to recognize specific hand poses so the effects trigger seamlessly.



Architecture & Requirements

Tools & Libraries

- **Python 3.11:** The main programming language for connecting the camera feed, math operations, and machine learning models.
- **OpenCV (cv2):** Handles grabbing webcam frames, drawing visual elements on screen, and rendering graphics overlay effects.
- **MediaPipe Task Vision:** Extracts hand joint locations in real time.
- **NumPy:** Used for vector math, calculating distances, and managing canvas array transformations.
- **Scikit-Learn:** Used to train a **Random Forest Classifier** to classify gestures based on hand landmark positions.

Concepts & Math Used

- **Wrist-Relative Coordinates (Position Invariance):** MediaPipe returns coordinates across the camera frame. If you move your hand to the corner of the screen, raw coordinates change completely even if your hand shape stays the same. To solve this, every landmark coordinate (x, y, z) is subtracted from the wrist coordinate (x_0, y_0, z_0) . This centers the origin at the wrist, ensuring gesture recognition stays identical no matter where your hand sits on screen.
- **Scale Normalization:** Hand size changes as you move closer to or further from the camera. The coordinates are scaled relative to the distance between key hand joints (like the palm width or wrist-to-middle-finger length) so gestures stay consistent at any distance.
- **Random Forest Classification:** Rather than training a heavy deep learning model directly on entire image pixels (which slows down performance and depends on lighting), the hand is simplified into 21 keypoint coordinates (63 total x, y, z values). A Random Forest model evaluates these numeric features through decision trees, delivering sub-millisecond inference speeds and keeping frame rates smooth.

How It Works (Execution Flow)

The application runs through a four-part workflow managed by a central control script:

1. Main Entry Point (**main.py**)

- Acts as the main console menu. It lets you select whether you want to collect new gesture data, train the machine learning model, launch the interactive application, or exit.

2. Feature Extraction (**geometry.py**)

- Whenever a frame is captured, MediaPipe detects 21 hand landmarks. **geometry.py** takes these raw coordinates, normalizes them relative to the wrist, scales them, and flattens them into a clean 63-element numeric list representing the current hand pose.

3. Data Collection (**collect_data.py**)

- When recording training samples, this script opens a camera feed, detects your hand, runs it through **geometry.py**, and appends the normalized coordinates along with a class label (e.g., Fist, Open Palm, Peace, Pinch) into a CSV file (**gestures_dataset.csv**).

4. Model Training & Pickling (**train_model.py**)

- Reads the CSV dataset and runs an automated split experiment loop. It tests different training/testing split ratios and random seeds to find the highest-performing Random Forest model. Once validated, it serializes (pickles) the trained model into a **.pkl** file so it can be loaded instantly without re-training.

5. Interactive Mode (**game.py**)

- Loads the trained **.pkl** model and opens the live camera feed. As you move your hand, it extracts keypoint features frame-by-frame, predicts your gesture, and immediately renders corresponding canvas effects and particle graphics over your hand.

Setup & How to Run

1. Set Up the Virtual Environment

Make sure Python 3.11 is active to maintain full compatibility with MediaPipe and OpenCV binaries:

```
python3.11 -m venv .venv  
source .venv/bin/activate
```

2. Install Dependencies

```
pip install opencv-python mediapipe numpy pandas scikit-learn
```

3. Run the Application

For macOS environment stability, pass the CPU flag when launching the main menu:

```
MEDIAPIPE_GPU=0 python main.py
```

4. Workflow inside the Menu:

- Select [1] to collect gesture data for your custom hand poses.
- Select [2] to run the automated split training loop and export `gesture_model.pkl`.
- Select [3] to launch the live generative canvas.

```
=====
HAND GESTURE CONTROLLED APPLICATION
=====
[1] Data Collector Mode
[2] Training Mode
[3] Game Mode
[4] Exit
=====
```

Issues Faced & Solutions

- **Python Version Conflicts on macOS:** macOS defaults to Python 3.13, which lacked pre-built wheel support for MediaPipe during setup.
 - **Fix:** Created an isolated Python 3.11 environment to ensure native binary support across libraries.
- **Apple Silicon Metal Delegate Crash:** MediaPipe's default C++ GPU delegate occasionally threw assertion failures ([DrishtiMetalHelper](#)) on macOS when initializing hardware pipelines.
 - **Fix:** Explicitly configured runtime environment variables (`MEDIAPIPE_GPU=0`) to handle CPU/GPU delegate context allocations smoothly.
- **Lighting and Positioning Sensitivity:** Raw screen coordinates caused predictions to fail if hand positions changed relative to the camera lens.
 - **Fix:** Implemented wrist-centered relative coordinate calculations in [geometry.py](#), removing lighting dependence and making gesture recognition invariant to frame positioning.

AI Assistance & Technical Lessons

AI tools were leveraged throughout this project as an accelerator for debugging C++ delegate crashes, structuring script boilerplate, and refining mathematical geometry calculations.

However, using AI highlights a clear boundary between generating code and understanding systems:

- **AI Makes Mistakes:** Language models frequently suggest outdated library syntax, misinterpret native hardware driver errors, or write fragile code that breaks on edge cases.
- **Understanding Core Concepts is Essential:** Without understanding relative coordinate transforms, matrix operations, or model validation metrics, it is impossible to debug runtime environment failures or improve model performance.
- **Vibe-Coding vs. True System Authoring:** Relying on AI without knowing how the underlying architecture works keeps you on the "user side"—stuck whenever the generated code breaks. To stay on the "author side," you must understand the data attributes, mathematical constraints, and execution flow so you can direct the tool effectively and build reliable software.